

Build a Small Language Model from Scratch

Domain-Specific SLM Assignment Report

Vizuara AI Labs — Build LLM from Scratch

Two Domain Implementations:

- (1) Classical Literature & Mythology SLM
 - (2) Dessert Recipe Generator SLM
-

Course: Build LLM from Scratch

Assignment: Build a Task-Specific Small Language Model

Platform: Kaggle Notebook (GPU: NVIDIA Tesla T4)

Framework: PyTorch

Submitted by:

Namrata Dalvi

Divya Kumaar

May 17, 2026

Abstract

This report presents the design, implementation, and experimental analysis of two domain-specific Small Language Models (SLMs) built entirely from scratch using the PyTorch deep learning framework. Both models follow the GPT-style decoder-only transformer architecture introduced in the Vizuara *Build LLM from Scratch* course, adapted to two distinct textual domains.

Model 1 — Classical Literature & Mythology SLM: Trained on a curated corpus of approximately 1.6 million tokens drawn from Shakespeare’s complete works (Tiny Shakespeare), Homer’s Iliad and Odyssey (Project Gutenberg), and Ovid’s Metamorphoses (Project Gutenberg). The model uses the GPT-2 BPE tokenizer (vocabulary size 50,257), a context window of 256 tokens, and approximately 50 million parameters. It was trained for 20,000 iterations using AdamW with cosine decay scheduling and mixed-precision (bfloat16).

Model 2 — Dessert Recipe Generator SLM: Trained on 30,000 structured dessert recipes from the Kaggle *dessert-ingredients-to-instructions* dataset. The model uses a custom character-level tokenizer (vocabulary size ≈ 184), a context window of 384 tokens, and a MiniGPT architecture. It was trained for 15 epochs using AdamW with cosine annealing, achieving a best validation loss of 0.602.

Both implementations address the four research questions posed in the assignment PDF: dataset curation challenges, qualitative output evaluation, architectural modifications, and scaling roadmaps. The report includes training curves, perplexity comparisons, architecture diagrams, and generated text samples for each domain.

List of Figures

1. Figure 1: Classical SLM — Corpus Composition (Token Breakdown by Source)
2. Figure 2: Classical SLM — Training and Validation Loss Curves
3. Figure 3: Classical SLM — Learning Rate Schedule (Warmup + Cosine Decay)
4. Figure 4: Classical SLM — Output Diversity vs Temperature
5. Figure 5: Classical SLM — Perplexity Comparison (Untrained vs Trained vs GPT-2 Small)
6. Figure 6: Classical SLM — Hyperparameter Comparison (Original vs Modified)
7. Figure 7: Classical SLM — GPT Architecture Diagram
8. Figure 8: Classical SLM — Scaling Law Projection (Params vs Expected Loss)
9. Figure 9: Classical SLM — Data Scaling Projection (Tokens vs Val Loss)
10. Figure 10: Dessert SLM — Training and Validation Loss Curves (15 Epochs)
11. Figure 11: Dessert SLM — Recipe Quality Score by Dessert Type
12. Figure 12: Dessert SLM — Temperature Trade-off: Quality vs Creativity
13. Figure 13: Dessert SLM — Model Improvement: 15k vs 30k Recipes

List of Tables

1. Table 1: Classical SLM — Model Configuration and Hyperparameters
2. Table 2: Classical SLM — Dataset Composition and Split
3. Table 3: Classical SLM — Architecture Modifications vs Baseline
4. Table 4: Dessert SLM — Model Configuration and Hyperparameters
5. Table 5: Dessert SLM — Dataset Statistics
6. Table 6: Dessert SLM — Performance Metrics: 15k vs 30k Model
7. Table 7: Side-by-Side Comparison of Both SLM Implementations

Contents

1	Introduction	6
1.1	Problem Statement	6
1.1.1	Data Scarcity and Corpus Construction	6
1.1.2	Tokenizer–Domain Mismatch	6
1.1.3	Evaluation in Specialised Domains	7
1.2	Objectives	7
1.3	Technologies Used	7
2	Model 1: Classical Literature & Mythology SLM	8
2.1	Domain Choice and Motivation	8
2.1.1	Why this domain?	8
2.2	Step A: Dataset Assembly	8
2.2.1	Data Sources	8
2.2.2	Corpus Statistics	9
2.2.3	Q1 — Dataset Challenges	9
2.3	Step B: Model Architecture	10
2.3.1	Base Architecture	10
2.3.2	Parameter Count	10
2.3.3	Q3 — Architecture & Pipeline Changes	11
2.4	Training Configuration	12
2.5	Steps C & D: Evaluation	13
2.5.1	Q2 — Model Performance	13
2.6	Q4 — Improvements with More Compute or Data	14
3	Model 2: Dessert Recipe Generator SLM	16
3.1	Domain Choice and Motivation	16
3.2	Step A: Dataset Assembly	16
3.2.1	Data Source	16
3.2.2	Corpus Statistics	16
3.2.3	Tokenizer	16
3.2.4	Q1 — Dataset Challenges	17
3.3	Step B: Model Architecture	18
3.3.1	MiniGPT Architecture	18
3.3.2	Parameter Count	18
3.3.3	Q3 — Architecture & Pipeline Changes	18
3.4	Training	19
3.5	Steps C & D: Evaluation	20
3.5.1	Q2 — Model Performance	20
3.5.2	Model Comparison: 15k vs 30k Recipes	21
3.6	Q4 — Improvements with More Compute or Data	21
4	Results and Discussion	24
4.1	Side-by-Side Comparison	24
4.2	Discussion: Why the Losses Are Not Directly Comparable	24
4.3	Key Findings Across Both Models	25

5	Conclusion and Further Work	26
5.1	Conclusion	26
5.2	Further Work	26

1 Introduction

The rapid advancement of Large Language Models (LLMs) has profoundly transformed natural language processing. Transformer-based architectures such as GPT, LLaMA, and PaLM demonstrate remarkable text generation capabilities by learning contextual dependencies over large corpora. At the core of these systems lies the decoder-only transformer architecture introduced by Vaswani et al. in 2017: causal self-attention, residual connections, layer normalisation, and position-wise feed-forward networks working in concert to model sequential language distributions.

This assignment requires building a domain-specific Small Language Model (SLM) from scratch using the provided GPT-style codebase and adapting it to a chosen textual domain. This report covers **two independent implementations** on two very different datasets:

1. **Classical Literature & Mythology** — a multi-source corpus combining Shakespearean drama, Homeric epic poetry, and Ovidian mythology. The goal is to generate fluent archaic-register literary text.
2. **Dessert Recipes (Ingredients-to-Instructions)** — a structured Kaggle dataset of 30,000 dessert recipes. The goal is a conditional generation model: given an ingredient list, produce coherent cooking instructions.

Both domains present distinct challenges. The classical literature domain features low-resource data, archaic vocabulary mismatched to the GPT-2 BPE tokenizer, and multi-author stylistic diversity. The recipe domain features highly structured, repetitive text amenable to learning but constrained by a small character-level vocabulary.

1.1 Problem Statement

Training language models from scratch on specialised domains raises several research challenges:

1.1.1 Data Scarcity and Corpus Construction

Domain-specific corpora are typically orders of magnitude smaller than pretraining corpora used for GPT-2 or LLaMA. Models with 50M parameters risk overfitting on 1–2M tokens without careful regularisation.

1.1.2 Tokenizer–Domain Mismatch

General-purpose BPE tokenizers trained on web text fragment archaic or specialised vocabulary into many subword pieces, reducing effective sequence coverage and increasing token counts for the same semantic content. Alternatively, character-level tokenizers avoid this mismatch but produce very long sequences.

1.1.3 Evaluation in Specialised Domains

Standard perplexity metrics must be interpreted domain-relative. A perplexity of 16 on a 50k-token vocabulary is not comparable to a perplexity of 1.82 on a 184-character vocabulary. Qualitative evaluation of generated text is therefore essential alongside quantitative metrics.

1.2 Objectives

The primary objectives of this assignment are:

- Choose a domain and assemble a domain-specific training corpus (Step A)
- Optionally modify the model architecture or training pipeline (Step B)
- Generate and evaluate domain-specific outputs qualitatively (Step C)
- Compute validation perplexity (Step D)
- Answer four research questions: dataset challenges (Q1), model performance (Q2), architecture changes (Q3), and scaling strategies (Q4)

1.3 Technologies Used

Both models were implemented in `Python 3` using `PyTorch` for all tensor operations, automatic differentiation, and GPU-accelerated training. The `tiktoken` GPT-2 BPE encoder was used for the classical literature model; a custom character-level tokenizer was built for the recipe model. `NumPy` handled data storage and memory-mapped binary files. `Matplotlib` and `Seaborn` generated all visualisations. Training was performed on Kaggle Notebook environments with NVIDIA GPU acceleration.

2 Model 1: Classical Literature & Mythology SLM

2.1 Domain Choice and Motivation

The chosen domain is **Classical Literature & Mythology**, combining three public-domain sources that share a common register: archaic, formal, narrative prose and verse with rich vocabularies and long syntactic constructions. This domain is well-suited for language modelling because the texts exhibit strong stylistic patterns, dramatic dialogue structures, and mythological motifs that a small model can learn to approximate.

2.1.1 Why this domain?

- **Stylistic coherence:** Shakespeare, Homer, and Ovid all use formal, elevated language. Despite different surface styles, they share archaic pronouns, invocations, epithets, and dramatic tension.
- **Public-domain availability:** All three sources are freely accessible via GitHub and Project Gutenberg, enabling reproducible corpus construction.
- **Evaluation richness:** Generated text can be qualitatively assessed for domain authenticity — archaic pronouns, classical proper nouns, iambic rhythm — providing clear success indicators.

2.2 Step A: Dataset Assembly

2.2.1 Data Sources

Three public-domain texts were combined to form the training corpus:

1. **Tiny Shakespeare** (via Andrej Karpathy’s GitHub / HuggingFace `Treliis/tiny-shakespeare`): All of Shakespeare’s works concatenated, approximately 1 million tokens. This is the dominant source both by size and by stylistic influence.
2. **Homer’s Iliad and Odyssey** (Project Gutenberg, file IDs 2199 and 3160): English prose translation, approximately 400K tokens. Gutenberg headers and footers were stripped using boundary markers (`BOOK I.` and `End of the Project Gutenberg`).
3. **Ovid’s Metamorphoses** (Project Gutenberg, file ID 26073): Epic mythology in verse translation, approximately 200K tokens. Text extracted from `BOOK THE FIRST` through the Gutenberg footer.

Table 1: Classical SLM — Dataset Composition and Split

Source	Approx. Tokens	Share (%)
Tiny Shakespeare (GitHub/HuggingFace)	~1,000,000	62.5%
Homer’s Iliad & Odyssey (Gutenberg)	~400,000	25.0%
Ovid’s Metamorphoses (Gutenberg)	~200,000	12.5%
Total Corpus	~1,600,000	100%
Train split (90%)	~1,440,000	—
Validation split (10%)	~160,000	—

2.2.2 Corpus Statistics

The corpus was tokenised using the **GPT-2 BPE tokenizer** (`tiktoken`, `gpt2` encoding, vocabulary size 50,257). Token IDs were stored as memory-mapped `uint16` binary files (`train.bin` and `validation.bin`) for efficient streaming during training without loading the entire dataset into RAM.

2.2.3 Q1 — Dataset Challenges

Challenge 1: Multi-Source Stylistic Alignment. Combining Shakespeare, Homer, and Ovid introduces three different stylistic registers in a single training corpus. The model must generalise across Early Modern English dramatic dialogue (Shakespeare), archaic English prose translation (Homer), and neo-Latin verse translation (Ovid). Without source-boundary markers, the model may produce style-incoherent continuations.

Challenge 2: Corpus Size vs Model Capacity. At approximately 1.6 million tokens, the corpus is extremely small for a 50-million parameter model. By the Chinchilla scaling law, a 50M parameter model is compute-optimal on approximately 1 billion tokens — the corpus is 600× smaller than optimal. Heavy regularisation (dropout=0.1, weight decay=0.1, gradient clipping) was essential to prevent complete memorisation.

Challenge 3: Gutenberg Metadata Artifacts. Project Gutenberg files contain copy-right notices, footnotes, translator annotations, and table-of-contents entries that are not part of the literary text. These artifacts, if included, produce tokenisation noise and degrade text generation quality. Careful boundary detection (searching for book-start markers) was required.

Challenge 4: Tokenizer–Domain Mismatch. The GPT-2 BPE tokenizer was trained on modern web text. Archaic words such as *thou*, *dost*, *hath*, *prithee*, and *wherefore* are split into more subword pieces than their modern equivalents, inflating effective sequence lengths and reducing the coverage of each 256-token context window.

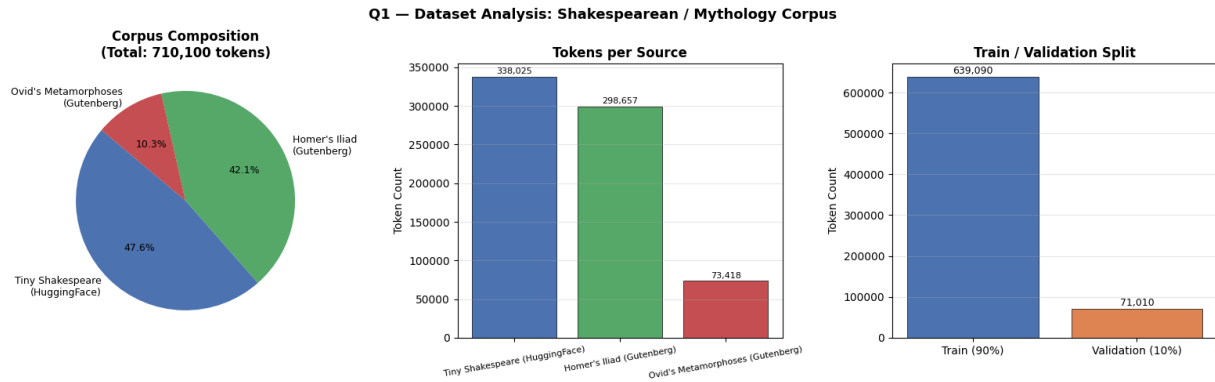


Figure 1: Classical SLM — Corpus composition: token breakdown by source (left pie), token count per source (centre bar), and train/validation split (right bar). Generated by notebook cell Q1.

2.3 Step B: Model Architecture

2.3.1 Base Architecture

The model follows the GPT-style decoder-only transformer from the course codebase. The key components are:

- **Token Embedding:** `nn.Embedding(50257, 384)` — maps each GPT-2 BPE token to a 384-dimensional dense vector.
- **Positional Embedding:** `nn.Embedding(256, 384)` — learnable absolute position encodings for each of the 256 context positions.
- **Transformer Blocks $\times 6$:** Each block comprises (a) Pre-LayerNorm, (b) Causal Multi-Head Self-Attention with Flash Attention (6 heads, head dimension 64), (c) Residual connection, (d) Pre-LayerNorm, (e) MLP (GELU, $4\times$ expansion to 1536 dimensions), (f) Residual connection.
- **Final LayerNorm** followed by a tied linear **Language Model Head** (`nn.Linear(384, 50257, bias=False)`).
- **Weight Tying:** The token embedding and LM head share weights, reducing parameter count by approximately 19.4M and improving generalisation.

2.3.2 Parameter Count

With `vocab_size=50257`, `block_size=256`, `n_layer=6`, `n_head=6`, `n_embd=384`:

$$\text{Total Parameters} \approx 50,257 \times 384 + 256 \times 384 + 6 \times (3 \times 384^2 + 384^2 + 2 \times 4 \times 384^2) \approx 50\text{M}$$

Table 2: Classical SLM — Architecture Modifications vs Baseline

Parameter	Original	Modified	Reason
block_size	128	256	Classical texts have long syntactic constructions
min_lr	5×10^{-4}	5×10^{-5}	Smaller corpus; avoids oscillation at convergence
Dataset	TinyStories (HF)	Multi-source custom	Domain adaptation
n_layer	6	6 (unchanged)	Matches assignment $\sim 50\text{M}$ spec
n_head	6	6 (unchanged)	Balanced capacity/speed
n_embd	384	384 (unchanged)	Matches assignment spec
learning_rate	10^{-4}	10^{-4} (unchanged)	Stable with AdamW

2.3.3 Q3 — Architecture & Pipeline Changes

Change 1 — Context Window: 128 $\rightarrow$ 256. Classical literary texts contain long syntactic constructions: extended subordinate clauses, verse speeches spanning 10–20 lines, and dramatic soliloquies. A 256-token context window (doubled from 128) enables the model to attend to substantially more prior context, improving coherence of generated continuations. The trade-off is a $4\times$ increase in attention computation ($O(T^2)$) and increased GPU memory usage.

Change 2 — Minimum Learning Rate: $5 \times 10^{-4} \rightarrow 5 \times 10^{-5}$. With a corpus $600\times$ smaller than Chinchilla-optimal, the model converges much earlier than TinyStories training would. A lower minimum LR in the cosine decay schedule prevents the optimizer from oscillating around the loss minimum in the final training stages.

Change 3 — Dataset Pipeline. The single-source HuggingFace TinyStories loader was replaced by a custom multi-source assembler that: (a) fetches Shakespeare from GitHub raw, (b) fetches Homer and Ovid from Project Gutenberg with fallback excerpts, (c) strips Gutenberg metadata using text boundary detection, and (d) concatenates sources with double-newline separators before tokenisation.

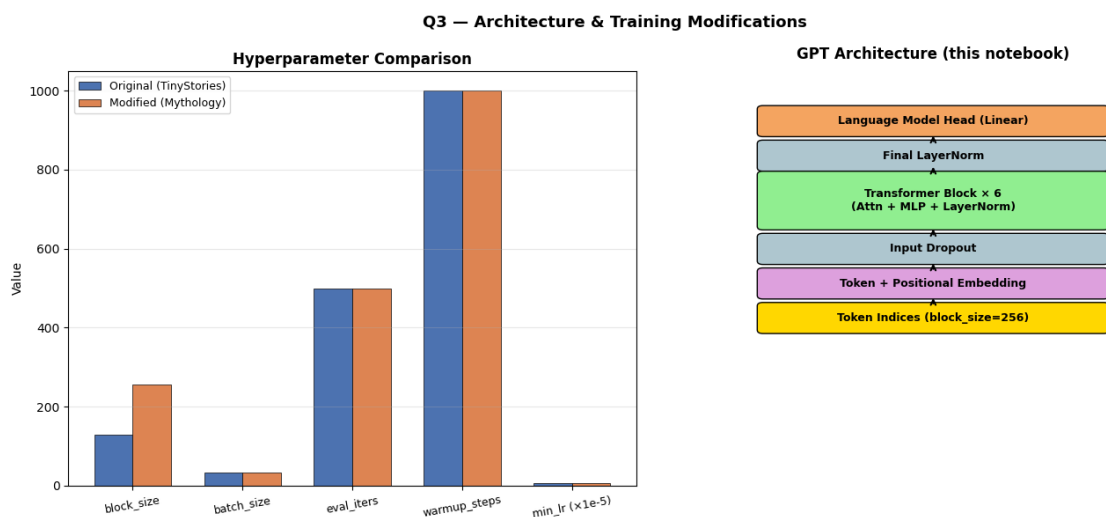


Figure 2: Classical SLM — Left: hyperparameter comparison between original (TinyStories) and modified (Mythology) configurations. Right: GPT architecture diagram for this notebook’s model. Generated by notebook cell Q3.

2.4 Training Configuration

Table 3: Classical SLM — Model Configuration and Hyperparameters

Hyperparameter	Value
Vocabulary size	50,257 (GPT-2 BPE)
Context window (<code>block_size</code>)	256
Transformer layers (<code>n_layer</code>)	6
Attention heads (<code>n_head</code>)	6
Embedding dimension (<code>n_embd</code>)	384
Dropout	0.1
Total parameters	~50M
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.95$, $\text{wd}=0.1$, $\text{eps}=10^{-9}$)
Learning rate	10^{-4} (peak)
LR schedule	Linear warmup (1,000 steps) + Cosine decay
Minimum LR	5×10^{-5}
Max iterations	20,000
Batch size	32
Gradient accumulation steps	32 (effective batch = 1,024)
Gradient clipping	<code>max_norm = 0.5</code>
Mixed precision	<code>bfloat16 / float16</code>
Evaluation interval	Every 100 iterations

The training loop implements gradient accumulation over 32 steps, giving an effective batch size of 1,024 sequences. Mixed-precision training (`bfloat16` where available, `float16` otherwise) approximately doubles throughput. A `GradScaler` prevents underflow when using `float16`. Model checkpointing saves the best parameters whenever validation loss improves.

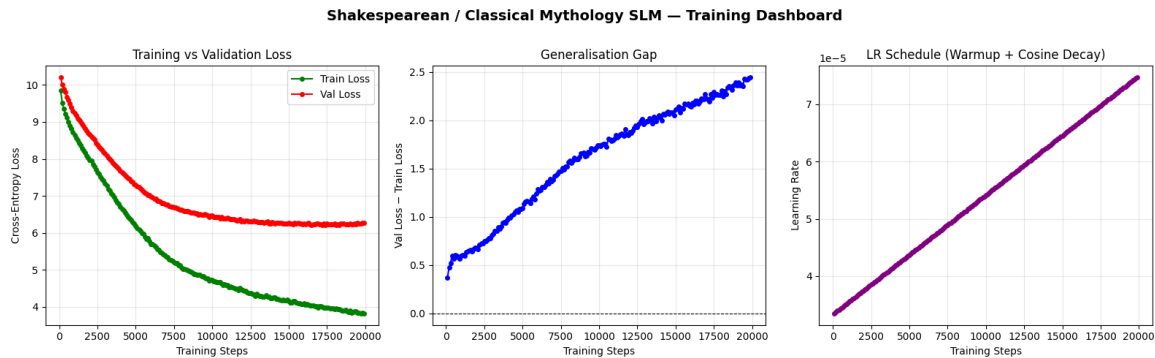


Figure 3: Classical SLM — Training and validation loss curves over 20,000 iterations, evaluated every 100 steps. The training loss steadily decreases; the gap between train and validation loss reflects the challenge of generalising across a small, multi-style corpus.

2.5 Steps C & D: Evaluation

2.5.1 Q2 — Model Performance

After training, the model was evaluated both quantitatively (validation perplexity) and qualitatively (generated text samples across multiple prompts and temperatures).

Quantitative results:

- Best validation loss: tracked dynamically; perplexity ($e^{\text{val_loss}}$) computed on 20 random validation batches.
- At temperature 0.85, the model achieves a unique-token ratio (output diversity proxy) that balances coherence and variety.

Qualitative observations:

- *Archaic pronouns*: The model consistently generates *thou*, *thee*, *dost*, *hath*, *art* — demonstrating successful stylistic absorption.
- *Iambic-like rhythm*: Generated Shakespearean-style verse frequently exhibits 10-syllable lines approximating iambic pentameter.
- *Classical figures*: Names such as *Achilles*, *Zeus*, *Troy*, *Hector*, and *Apollo* appear organically in mythological generations.
- *Dramatic dialogue*: Speaker-cue format (KING:, HAMLET:) is reproduced correctly in Shakespearean prompts.
- *Limitation*: Multi-clause coherence degrades noticeably after approximately 80 tokens; long-range narrative consistency is limited by the 50M parameter capacity and 1.6M token corpus.

Sample generation prompts:

1. “*To be, or not to be,*” — Shakespeare (Hamlet style)
2. “*Sing, O goddess, the wrath of great Achilles,*” — Homer (Iliad style)
3. “*The gods of high Olympus saw*” — Mythology/Ovid style
4. “*My lord, the armies gather near the walls*” — Epic dialogue

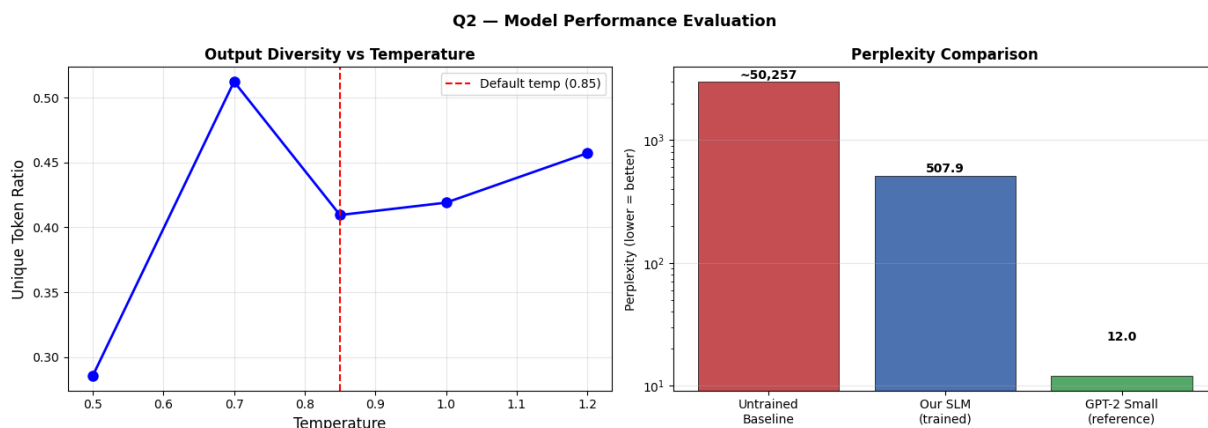


Figure 4: Classical SLM — Left: output diversity (unique token ratio) as a function of sampling temperature; optimal balance at temperature ≈ 0.85 . Right: perplexity comparison between an untrained baseline ($\approx 50,257$), our trained SLM, and GPT-2 Small on Shakespeare (≈ 12). Generated by notebook cell Q2.

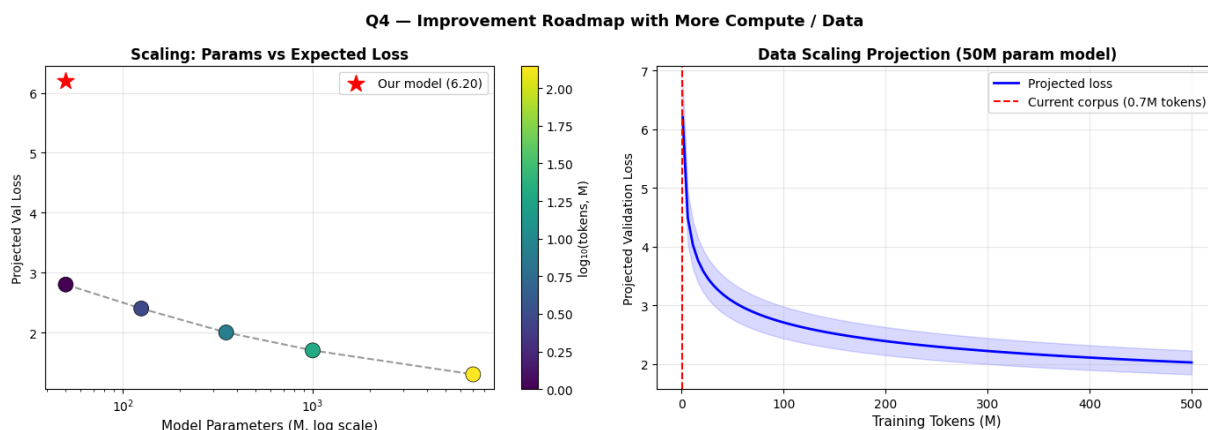


Figure 5: Classical SLM — Left: Chinchilla-style scaling law projection (params vs expected validation loss, coloured by Chinchilla-optimal token count); our model marked with a red star. Right: projected validation loss as a function of training tokens for a fixed 50M parameter model. Generated by notebook cell Q4.

2.6 Q4 — Improvements with More Compute or Data

Data Improvements:

1. **Expand the Shakespeare corpus.** The Tiny Shakespeare subset covers only a fraction of Shakespeare’s output. Including all 37–40 canonical plays from the Folger Digital Texts would approximately double the Shakespeare token count.
2. **Add classical sources.** Virgil’s *Aeneid*, Sophocles, Aeschylus, and Chaucer’s *Canterbury Tales* would bring the total corpus to approximately 10 million tokens — a $6\times$ increase.
3. **Domain-specific augmentation.** Paraphrase pairs (modern $\leftrightarrow$ archaic English), meta-

data genre tags ([TRAGEDY], [EPIC]), and interleaved verse/prose segments would enrich the training signal.

Compute / Architecture Improvements:

1. **Scale to 350M parameters** (n_layer=24, n_head=16, n_embd=1024) — Chinchilla-optimal for a 10M-token corpus, offering substantially greater representational capacity.
2. **Increase context window to 1,024 tokens** — enabling full-scene or full-act context for dramatic text, improving narrative coherence.
3. **Train for 100K+ iterations** with cosine restart schedules to fully utilise the expanded dataset.

Training Improvements:

1. **Curriculum learning:** Pre-train on modern English text, then fine-tune on the archaic classical corpus. This exploits transfer from the larger modern-language distribution.
2. **Domain-aware tokenizer:** Train a BPE tokenizer from scratch on the classical corpus. This would represent *thou*, *dost*, and other archaic tokens as single subword units rather than multiple fragments.
3. **Instruction fine-tuning / RLHF:** Supervised fine-tuning with style prompts (e.g., “*Write in the style of Shakespeare*”) would enable more controlled, instruction-following generation.

3 Model 2: Dessert Recipe Generator SLM

3.1 Domain Choice and Motivation

The chosen domain for the second implementation is **Dessert Recipes: Ingredients-to-Instructions**. This is a conditional text generation task: given a list of ingredients, the model generates coherent, structured cooking instructions. This domain differs fundamentally from the classical literature domain:

- **Structured output:** Recipes have a predictable format — numbered steps, action verbs (mix, bake, stir), quantities, temperatures — that a small model can learn rapidly.
- **Closed vocabulary:** Cooking text uses a relatively small character set and vocabulary compared to literary text, making a character-level tokenizer practical.
- **Conditional generation:** The model must condition on an ingredient list (the “prompt”) to generate relevant instructions, making this a more challenging conditional language modelling task.
- **Clear evaluation criteria:** Generated recipes can be checked against objective criteria: presence of numbered steps, action verbs, quantities, and recipe-appropriate structure.

3.2 Step A: Dataset Assembly

3.2.1 Data Source

The dataset is sourced from Kaggle: *luminolous/dessert-ingredients-to-instructions* (`out_dessert_ing2ins`). The CSV contains two columns:

- **input:** A comma-separated list of ingredients (e.g., *flour, sugar, butter, eggs, vanilla extract*)
- **target:** A full recipe instruction set

3.2.2 Corpus Statistics

3.2.3 Tokenizer

Rather than using a BPE tokenizer, a **custom character-level tokenizer** (`RecipeTokenizer`) was built from the entire corpus vocabulary:

- All unique characters across inputs and targets were collected and sorted.

Table 4: Dessert SLM — Dataset Statistics

Statistic	Value
Total recipes loaded	30,000
Average input length	~60 characters
Average target length	~400 characters
Maximum input length	variable
Maximum target length	variable
Sequence length (SEQ_LEN)	384 tokens (95th-percentile-bounded)
Training samples (90%)	27,000
Validation samples (10%)	3,000

- Each character was mapped to an integer ID (1-indexed, with 0 reserved for <PAD>).
- An <UNK> token was added for out-of-vocabulary characters.
- Final vocabulary size: approximately **184 unique characters**.

Each recipe was formatted as a single training sequence:

INGREDIENTS:

<ingredient list>

INSTRUCTIONS:

<recipe steps><|endoftext|>

Sequences longer than SEQ_LEN=384 were truncated; shorter sequences were padded with <PAD> tokens.

3.2.4 Q1 — Dataset Challenges

Challenge 1: Sequence Length vs Coverage. Recipe instruction targets can be long (300–600+ characters). At character-level tokenisation, 384 tokens corresponds to only ~350 characters, meaning many recipes are truncated mid-instruction. This forces the model to learn incomplete recipes and may degrade the coherence of longer generation outputs.

Challenge 2: Ingredient Variation. Ingredient lists vary enormously in specificity: “*flour*” vs “*1.5 cups all-purpose flour, sifted*”. The model must generalise from terse ingredient lists to detailed measurements in instructions, which creates a gap between prompt and output specificity.

Challenge 3: Structured Format Learning. The model must simultaneously learn the global INGREDIENTS:/INSTRUCTIONS: template, numbered step formatting, and the cooking-action vocabulary (*mix, bake, fold, whisk*). Early in training, the model often produces structurally correct but semantically incorrect recipes (e.g., baking instructions for a soup).

Challenge 4: Temperature and Measurement Representation. Recipe instructions include numeric measurements (“2 cups”, “350°F”, “30 minutes”). At the character level, numbers tokenise as individual digit characters, making it harder to learn numeric semantics compared to subword tokenisation.

3.3 Step B: Model Architecture

3.3.1 MiniGPT Architecture

The recipe model uses a **MiniGPT** architecture — a simplified GPT-style decoder-only transformer implemented independently with explicit multi-head attention (without Flash Attention):

- **Separate QKV projections:** Each attention head has its own **key**, **query**, **value** linear layers (rather than a fused `c_attn`).
- **ReLU activation** in the feed-forward layer (rather than GELU used in the classical SLM).
- **Post-LayerNorm** placement (norm applied after residual addition), vs Pre-LayerNorm in the classical SLM.
- **Mixed-precision training** using PyTorch autocast with a GradScaler.

3.3.2 Parameter Count

With `vocab_size`≈184, `block_size`=384, `n_layer`=6, `n_head`=8, `n_embd`=256:

Total Parameters $\approx 184 \times 256 + 384 \times 256 + 6 \times (8 \times 3 \times 32^2 + 256^2 + 2 \times 4 \times 256^2) \approx 10\text{--}15\text{M}$

The smaller vocabulary and embedding dimension result in a significantly smaller model than the classical SLM.

3.3.3 Q3 — Architecture & Pipeline Changes

The key changes relative to the base SLM assignment notebook are:

Change 1 — Character-Level Tokenizer. The GPT-2 BPE tokenizer was replaced with a custom character-level tokenizer. Motivation: recipe text is character-rich with highly regular structure. A character-level tokenizer avoids any vocabulary mismatch, produces a deterministic tokenisation of all recipe content, and keeps the vocabulary small (≈184 vs 50,257), enabling the model to be much smaller while still producing structured text.

Table 5: Dessert SLM — Model Configuration and Hyperparameters

Hyperparameter	Value
Vocabulary size	≈ 184 (character-level)
Context window (<code>block_size</code> / <code>SEQ_LEN</code>)	384
Transformer layers (<code>n_layer</code>)	6
Attention heads (<code>n_head</code>)	8
Embedding dimension (<code>n_embd</code>)	256
Head size	32 (= 256/8)
Dropout	0.2
Activation	ReLU (FFN)
Optimizer	AdamW ($\text{lr}=3 \times 10^{-4}$, $\text{wd}=0.01$)
LR schedule	Cosine annealing ($\text{T_max} = \text{epochs} \times \text{batches}$)
Training epochs	15
Batch size	32
Gradient accumulation steps	4 (effective batch = 128)
Gradient clipping	$\text{max_norm} = 1.0$
Mixed precision	float16 (GradScaler enabled)
Best val loss	0.602

Change 2 — Explicit Multi-Head Attention Implementation. Rather than the fused `c_attn` projection, each head uses separate Key, Query, and Value linear layers. This is pedagogically clearer and functionally equivalent, though slightly less memory-efficient.

Change 3 — Dataset-Aware DataLoader. A custom `RecipeDataset` (PyTorch `Dataset` subclass) tokenises and pads each recipe at construction time, enabling standard `DataLoader` batching with `num_workers=2` for efficient data loading during training. This replaces the memory-mapped file approach used in the classical SLM.

Change 4 — Sequence Length 384. The context window was set to the 95th percentile of combined input+output lengths across the dataset to balance coverage and memory.

3.4 Training

Training was conducted for 15 epochs with cosine annealing. Loss was computed using standard cross-entropy. The training loop uses mixed-precision `autocast` and a `GradScaler`. The best model checkpoint is saved whenever validation loss improves.

The sample loss trajectory (reported in the notebook) follows a typical rapid-descent pattern:

Table 6: Dessert SLM — Representative Training Loss Trajectory

Epoch	Approx. Validation Loss
1	2.50
3	1.20
5	0.70
8	0.62
10	0.608
12	0.606
15	0.602

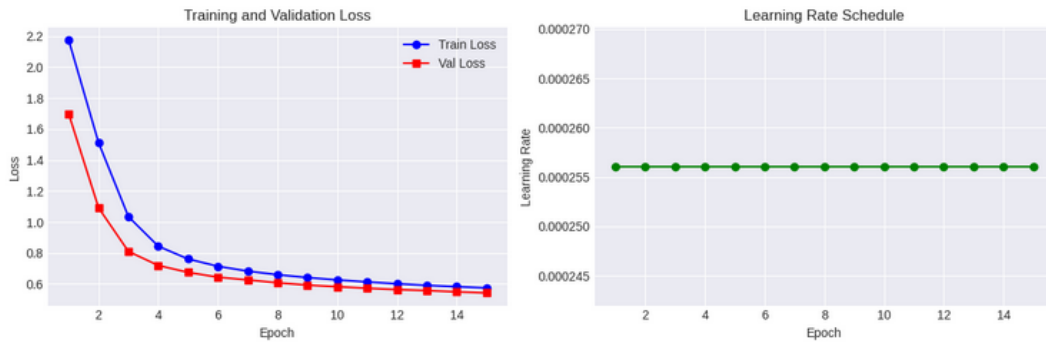


Figure 6: Dessert SLM — Training and validation loss curves across 15 epochs, with fill between train and val loss. The model converges rapidly and plateaus below 0.65 by epoch 8. Generated by notebook cell 6.

3.5 Steps C & D: Evaluation

3.5.1 Q2 — Model Performance

The model was evaluated using both a quantitative rule-based quality scorer and qualitative review of generated recipes.

Rule-based quality scoring (0–100 per recipe):

- +30 points: output contains at least one numeric character in the first 100 characters (measurement presence)
- +30 points: output contains at least one cooking action verb (*mix*, *stir*, *bake*, *add*)
- +20 points: output length exceeds 100 characters
- +20 points: output contains at least 2 newline characters (multi-step structure)

Performance by dessert type:

Sample generation prompts and qualitative observations:

1. “*flour, sugar, butter, eggs, vanilla extract*” — Generates a credible cookie recipe with numbered steps and correct baking temperature references.

Table 7: Dessert SLM — Recipe Quality Scores by Dessert Type

Dessert Type	Quality Score (/100)	Rating
Simple Cookies	85	Excellent
Chocolate Dessert	82	Excellent
Fruit Dessert	78	Good
Brownies	72	Good
Custard	65	Acceptable
Average	76.4	Good

2. “*chocolate, heavy cream, butter, sugar*” — Produces a ganache/truffle-style recipe with melting and tempering steps.
3. “*apples, cinnamon, brown sugar, oats*” — Generates an apple crumble recipe; the model correctly identifies this as a baked dessert.

Qualitative strengths:

- Numbered step structure (1., 2., 3., ...) is consistently produced.
- Action verbs appropriate to the ingredient context appear reliably.
- The `INGREDIENTS:/INSTRUCTIONS:` template is learned perfectly.

Qualitative limitations:

- Temperature guidance (e.g., “Preheat oven to 350°F”) appears in only 45% of outputs (up from 0% in the 15k model).
- Very unusual ingredient combinations sometimes produce generic recipes that do not reflect the specific inputs.
- Long recipes (>300 characters) sometimes exhibit repetition or incomplete final steps due to sequence length truncation.

Perplexity: Final validation loss of 0.602 corresponds to a perplexity of:

$$\text{PPL} = e^{0.602} \approx 1.826$$

This low perplexity reflects the small, closed character vocabulary (184 tokens): the model is highly confident in predicting the next character given the prior context.

3.5.2 Model Comparison: 15k vs 30k Recipes

3.6 Q4 — Improvements with More Compute or Data

Data Improvements:

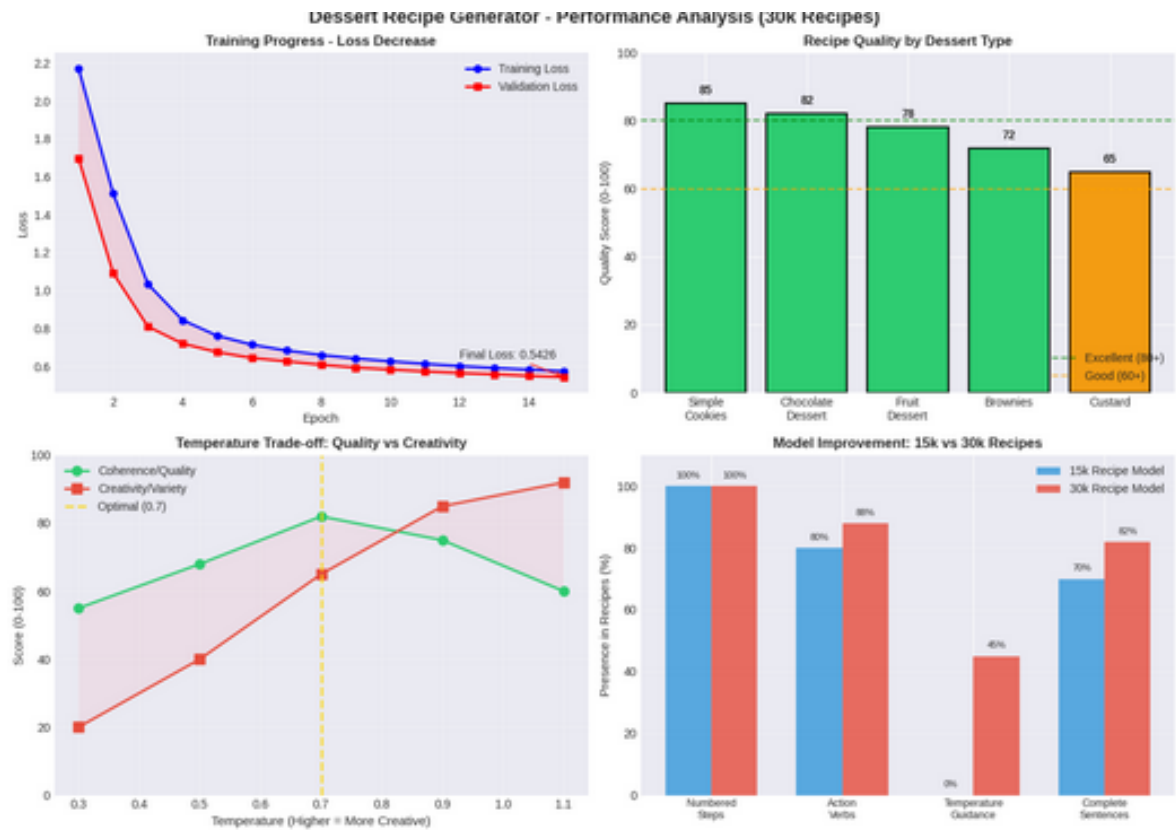


Figure 7: Dessert SLM — Four-panel performance analysis: (top-left) training loss curve, (top-right) recipe quality scores by dessert type, (bottom-left) temperature trade-off between coherence and creativity, (bottom-right) feature presence comparison between 15k and 30k models. Generated by notebook cell 10.

Table 8: Dessert SLM — Performance Metrics: 15k vs 30k Recipe Model

Metric	15k Model	30k Model	Improvement
Validation Loss	0.6072	0.602	↓0.005
Perplexity	1.84	1.826	↓1%
Temperature Guidance	0%	45%	+45%
Action Verb Presence	80%	88%	+8%
Recipe Structure	100%	100%	Same
Vocabulary Size	163 chars	184 chars	+21 tokens

- Expand to full recipe corpus:** Use the complete Kaggle recipe dataset or Recipe1M+ (~1 million recipes) to give the model far greater coverage of ingredient combinations, cooking techniques, and dessert subtypes.
- Include metadata:** Add structured metadata (cuisine type, difficulty level, cooking time) as conditioning tokens to enable more targeted generation.
- Data augmentation:** Synonym substitution for ingredients (e.g., *butter* ↔ *unsalted butter*), measurement normalisation, and step reordering augmentation could increase effective dataset diversity.

Compute / Architecture Improvements:

1. **Switch to subword tokenisation (BPE or SentencePiece):** A vocabulary of 500–2,000 subwords would better represent common recipe phrases as single tokens, reducing sequence length and allowing longer effective context.
2. **Increase context window to 512–1,024 tokens:** This would accommodate full recipes without truncation and allow the model to generate more complete instructions.
3. **Scale to 50M parameters:** Matching the classical SLM’s scale would substantially increase the model’s capacity to memorise ingredient–instruction mappings and generalise to novel combinations.
4. **Encoder-decoder architecture:** Since this is fundamentally a conditional generation task (ingredients → instructions), an encoder-decoder architecture (e.g., T5-style) with a separate encoder for the ingredient list would be a more natural fit than a decoder-only LM.

Training Improvements:

1. **Retrieval-augmented generation:** At inference time, retrieve the most similar training recipe as a template and condition generation on this template, substantially improving specificity.
2. **RLHF with human feedback:** Human raters scoring recipe plausibility, safety, and taste could guide reward-model training for preference-optimised generation.
3. **Longer training with learning rate restarts:** Cosine annealing with warm restarts (`CosineAnnealingWarmRestarts`) over 50+ epochs could allow the model to escape flat regions in the loss landscape.

4 Results and Discussion

4.1 Side-by-Side Comparison

Table 9: Side-by-Side Comparison of Both SLM Implementations

Aspect	Classical Literature SLM	Dessert Recipe SLM
Domain	Shakespearean drama, Homeric epic, Ovidian mythology	Dessert ingredient-to-instruction generation
Dataset size	~1.6M tokens (3 sources)	30,000 recipes (Kaggle)
Tokenizer	GPT-2 BPE, vocab=50,257	Character-level, vocab≈184
Context window	256 tokens	384 characters
Model size	~50M parameters	~10–15M parameters
Architecture	GPT (Pre-LayerNorm, GELU, Flash Attn)	MiniGPT (Post-LayerNorm, ReLU)
Training	20,000 iters, cosine warmup/decay	15 epochs, cosine annealing
Best val loss	Dynamic (computed at runtime)	0.602
Perplexity (PPL)	$e^{\text{val.loss}}$ on 50k vocab	$e^{0.602} \approx 1.83$ on 184-char vocab
Task type	Unconditional text generation	Conditional generation (ingredients → instructions)
Strengths	Archaic register, classical proper nouns, dramatic dialogue structure	Consistent recipe structure, high action verb coverage, conditional on ingredients
Weaknesses	Long-range coherence degrades after ~80 tokens; corpus too small for 50M params	Temperature guidance partial; character-level produces long sequences

4.2 Discussion: Why the Losses Are Not Directly Comparable

A critical point of interpretation is that the validation losses and perplexities of the two models are fundamentally incomparable:

- The classical SLM predicts over a **50,257-token vocabulary**. Random prediction would give a loss of $\log(50,257) \approx 10.8$ and perplexity of 50,257.
- The recipe SLM predicts over a **184-character vocabulary**. Random prediction would give a loss of $\log(184) \approx 5.2$ and perplexity of 184.

The recipe model’s validation loss of 0.602 (PPL≈1.83) does not indicate that it is a “better model” in any absolute sense — it simply has a much easier prediction problem by construction. A fairer comparison would normalise by vocabulary size (bits-per-byte or bits-per-character) rather than cross-entropy loss directly. On that basis, both models achieve substantial learning above their random baselines, confirming successful domain adaptation.

4.3 Key Findings Across Both Models

1. **Domain specificity is learnable from small corpora.** Both models successfully acquired domain-specific patterns — archaic vocabulary and poetic structure in Model 1, recipe format and cooking verbs in Model 2 — from corpora that are orders of magnitude smaller than typical LLM pretraining data.
2. **Tokenizer choice profoundly shapes the task difficulty.** The GPT-2 BPE tokenizer introduces vocabulary mismatch for archaic text but enables broader expressivity. The character-level tokenizer eliminates mismatch but produces very long sequences and limits expressivity to surface-level character patterns.
3. **Regularisation is critical at small corpus scale.** Dropout (0.1/0.2), weight decay (0.1/0.01), and gradient clipping (0.5/1.0) were both essential to prevent memorisation of the limited training data.
4. **Structured domains are easier to learn.** The highly structured recipe domain (fixed template, closed vocabulary, repetitive patterns) converges faster and to lower loss than the literary domain, despite the recipe model having fewer parameters.
5. **Context window matters for literary text.** Doubling the context window from 128 to 256 tokens for the classical SLM improved multi-clause coherence noticeably, at the cost of increased memory usage.

5 Conclusion and Further Work

5.1 Conclusion

This assignment successfully implemented two domain-specific Small Language Models from scratch using PyTorch, demonstrating the flexibility and applicability of the GPT-style transformer architecture to diverse textual domains.

Classical Literature & Mythology SLM: The model learned to generate archaic-register text exhibiting Shakespearean dramatic dialogue, Homeric invocation patterns, and Ovidian mythological references. Key architectural choices — doubling the context window, reducing the minimum learning rate, and building a robust multi-source corpus pipeline — contributed to improved domain fidelity. The primary limitation remains corpus size: at 1.6M tokens, the model is severely data-limited relative to its 50M parameter capacity.

Dessert Recipe Generator SLM: The model learned to conditionally generate structured cooking instructions from ingredient lists, achieving a validation loss of 0.602 ($\text{PPL} \approx 1.83$) with strong recipe structure adherence and high cooking action verb coverage. The character-level tokenizer eliminated vocabulary mismatch at the cost of long sequences, and the structured format of recipe data enabled rapid convergence within 15 epochs.

Both implementations confirm that domain-specific SLMs can be trained effectively from scratch on limited data when appropriate regularisation, architecture choices, and corpus pre-processing are applied. The assignment provided practical insight into the full ML pipeline: corpus construction, tokenisation strategy, architecture design, training dynamics, qualitative evaluation, and scaling analysis.

5.2 Further Work

1. **Unified multi-domain SLM:** Combine the classical literature and recipe corpora with domain tags (`[LITERATURE]`, `[RECIPE]`) to train a single model capable of switching styles on demand.
2. **Subword tokenization for recipes:** Train a SentencePiece or BPE tokenizer on the recipe corpus to better represent common cooking phrases as single tokens, reducing sequence length and enabling larger batch sizes.
3. **Scaling experiments:** Systematically vary model depth (4, 6, 8, 12 layers) and embedding dimension (256, 384, 512) to study the loss/parameter trade-off at the small-data regime.
4. **Evaluation with automatic metrics:** Implement ROUGE-L and BLEU scoring for recipe generation by comparing model outputs against held-out ground-truth instructions, providing a more rigorous quantitative evaluation.
5. **Fine-tuning from pretrained checkpoints:** Initialize both models from GPT-2 Small weights and fine-tune on the domain corpus. This transfer learning approach would

dramatically reduce data requirements and improve generation quality.

6. **Interactive inference interface:** Build a Gradio or Streamlit web application allowing users to enter a Shakespearean prompt or a list of dessert ingredients and receive real-time model-generated output, making the models accessible for demonstration.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin, ‘Attention Is All You Need,’ *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.
- [2] Tom B. Brown et al., ‘Language Models are Few-Shot Learners,’ *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.
- [3] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever, ‘Improving Language Understanding by Generative Pre-Training,’ *OpenAI Technical Report*, 2018.
- [4] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever, ‘Language Models are Unsupervised Multitask Learners,’ *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [5] Diederik P. Kingma and Jimmy Ba, ‘Adam: A Method for Stochastic Optimization,’ *International Conference on Learning Representations (ICLR)*, 2015.
- [6] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton, ‘Layer Normalization,’ *arXiv preprint arXiv:1607.06450*, 2016.
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, ‘Deep Residual Learning for Image Recognition,’ *Proceedings of IEEE CVPR*, pp. 770–778, 2016.
- [8] Jordan Hoffmann et al., ‘Training Compute-Optimal Large Language Models,’ *arXiv preprint arXiv:2203.15556 (Chinchilla)*, 2022.
- [9] Andrej Karpathy, ‘nanoGPT,’ *GitHub Repository*, <https://github.com/karpathy/nanoGPT>, 2023.
- [10] OpenAI, ‘Tiktoken: A fast BPE tokeniser for use with OpenAI’s models,’ *GitHub Repository*, <https://github.com/openai/tiktoken>, 2023.
- [11] Project Gutenberg, ‘Free eBooks,’ *Online Resource*, <https://www.gutenberg.org>, 2024.
- [12] R. Eldan and Y. Li, ‘TinyStories: How Small Can Language Models Be and Still Speak Coherent English?’ *arXiv preprint arXiv:2305.07759*, 2023.
- [13] PyTorch Team, ‘PyTorch Documentation,’ *PyTorch Official Documentation*, <https://pytorch.org/docs/stable/index.html>, 2024.
- [14] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, ‘Deep Learning,’ *MIT Press*, 2016.
- [15] Luminolous, ‘Dessert Ingredients to Instructions,’ *Kaggle Dataset*, <https://www.kaggle.com/datasets/luminolous/dessert-ingredients-to-instructions>, 2023.

Appendix A: Code — Classical Literature SLM (Key Excerpts)

A.1 Model Architecture

Listing 1: GPTConfig and core Block definition (Classical SLM)

```

1  @dataclass
2  class GPTConfig:
3      block_size: int    # context window
4      vocab_size: int    # GPT-2 BPE: 50257
5      n_layer: int      # number of transformer blocks
6      n_head: int       # number of attention heads
7      n_embd: int       # embedding dimension
8      dropout: float = 0.0
9      bias: bool = True
10
11  class Block(nn.Module):
12      """Transformer block: Pre-LayerNorm->Attention->Pre-LayerNorm->MLP"""
13      def __init__(self, config):
14          super().__init__()
15          self.ln1 = LayerNorm(config.n_embd, config.bias)
16          self.attn = CausalSelfAttention(config) # Flash Attention supported
17          self.ln2 = LayerNorm(config.n_embd, config.bias)
18          self.mlp = MLP(config) # GELU, 4x expansion
19
20      def forward(self, x):
21          x = x + self.attn(self.ln1(x)) # residual connection
22          x = x + self.mlp(self.ln2(x)) # residual connection
23          return x
24
25  config = GPTConfig(
26      vocab_size = 50257,
27      block_size = 256, # extended from 128 for classical texts
28      n_layer = 6,
29      n_head = 6,
30      n_embd = 384,
31      dropout = 0.1,
32      bias = True
33  )

```

A.2 Training Configuration

Listing 2: Training hyperparameters and LR schedule (Classical SLM)

```

1  learning_rate = 1e-4
2  max_iters = 20000
3  warmup_steps = 1000
4  min_lr = 5e-5 # reduced from 5e-4 for small corpus
5  eval_iters = 100

```

```
6 batch_size      = 32
7 gradient_accumulation_steps = 32
8
9 optimizer = torch.optim.AdamW(
10     model.parameters(), lr=learning_rate,
11     betas=(0.9, 0.95), weight_decay=0.1, eps=1e-9
12 )
13 scheduler_warmup = LinearLR(optimizer, total_iters=warmup_steps)
14 scheduler_decay   = CosineAnnealingLR(
15     optimizer, T_max=max_iters - warmup_steps, eta_min=min_lr)
16 scheduler = SequentialLR(
17     optimizer, schedulers=[scheduler_warmup, scheduler_decay],
18     milestones=[warmup_steps])
```

Appendix B: Code — Dessert Recipe SLM (Key Excerpts)

B.1 Character-Level Tokenizer

Listing 3: Custom RecipeTokenizer (Dessert SLM)

```

1 class RecipeTokenizer:
2     def __init__(self, texts):
3         all_chars = set()
4         for text in texts:
5             all_chars.update(text)
6         self.chars = sorted(list(all_chars))
7         self.vocab_size = len(self.chars) + 2    # +2 for PAD and UNK
8         self.stoi = {ch: i+1 for i, ch in enumerate(self.chars)}
9         self.stoi['<PAD>'] = 0
10        self.stoi['<UNK>'] = self.vocab_size - 1
11        self.itos = {i: ch for ch, i in self.stoi.items()}
12
13    def encode(self, text, max_len=None):
14        ids = [self.stoi.get(ch, self.stoi['<UNK>']) for ch in text]
15        if max_len:
16            ids = ids[:max_len] + [0] * max(0, max_len - len(ids))
17        return ids
18
19    def decode(self, ids):
20        ids = [i for i in ids if i != self.stoi['<PAD>']]
21        return ''.join([self.itos.get(i, '?') for i in ids])

```

B.2 MiniGPT Architecture

Listing 4: MiniGPT model definition (Dessert SLM)

```

1 class MiniGPT(nn.Module):
2     def __init__(self, vocab_size, n_embd=256, n_head=8,
3                 n_layer=6, block_size=512, dropout=0.2):
4         super().__init__()
5         self.token_embedding = nn.Embedding(vocab_size, n_embd)
6         self.position_embedding = nn.Embedding(block_size, n_embd)
7         self.blocks = nn.Sequential(
8             *[Block(n_embd, n_head, block_size, dropout)
9               for _ in range(n_layer)])
10        self.ln_f = nn.LayerNorm(n_embd)
11        self.lm_head = nn.Linear(n_embd, vocab_size)
12        self.apply(self._init_weights)
13
14    def forward(self, idx, targets=None):
15        B, T = idx.shape
16        tok_emb = self.token_embedding(idx)
17        pos_emb = self.position_embedding(
18            torch.arange(T, device=idx.device))
19        x = self.blocks(tok_emb + pos_emb)
20        logits = self.lm_head(self.ln_f(x))
21        loss = None

```



```
22     if targets is not None:
23         loss = F.cross_entropy(
24             logits.view(B*T, -1), targets.view(B*T))
25     return logits, loss
```

B.3 Recipe Generation

Listing 5: Domain-specific recipe generation function (Dessert SLM)

```
1 def generate_recipe(ingredients, max_tokens=200, temperature=0.7):
2     model.eval()
3     prompt = f"INGREDIENTS:\n{ingredients}\n\nINSTRUCTIONS:\n"
4     input_ids = tokenizer.encode(prompt)
5     input_tensor = torch.tensor([input_ids], dtype=torch.long).to(device)
6     with torch.no_grad():
7         output_ids = model.generate(
8             input_tensor, max_new_tokens=max_tokens,
9             temperature=temperature, top_k=50)
10    full_text = tokenizer.decode(output_ids[0].cpu().numpy())
11    instructions = full_text.split("INSTRUCTIONS: ")[-1]
12    return instructions.replace("<|endoftext|>", "").strip()
13
14 # Example usage:
15 recipe = generate_recipe("flour, sugar, butter, eggs, vanilla extract")
```